



Engenharia da Computação

Sistemas Operacionais

Prof.Felipe Viel

Luiz Gustavo, Rafael Barbosa

Relatório M2 – MicroKernel

2026/06

Sumário

1. INTRODUÇÃO.....	3
1.1 Materiais utilizados.....	3
2. DESENVOLVIMENTO.....	4
2.1 Arquivos adicionados.....	4
2.2 Arquivos modificados.....	4
3. IMPLEMENTAÇÃO.....	5
3.1. Módulo Timer.....	5
3.1.1. Arquivo timer.h.....	5
3.1.2. Chamada SBI timer.....	6
3.1.3. void timer_next(void).....	8
3.1.4. void timer_init(void).....	9
3.2. Tratamento de Traps.....	11
3.2.1. Arquivo trap.h.....	12
3.2.2. Arquivo trap.c.....	12
3.2.3. Leitura do CSR scause via Hardware.....	13
3.2.4. Decodificação de Causa (Bitmasking).....	13
3.2.5. Filtro e Tratamento do Timer do Supervisor (STIP).....	14
3.2.6. Tratamento de Falhas Críticas (Fallback).....	14
3.3 Ponto de Entrada em Assembly.....	15
3.3.1. Alocação do Trap Frame na Pilha.....	15
3.3.2. Salvamento do Stack Pointer Antigo.....	15
3.3.3. Salvamento em Massa dos Registradores Gerais.....	16
3.3.4. Despacho para o Ambiente C.....	16
3.3.5. Restauração de Contexto.....	16
3.3.6. Restauração do Ponteiro da Pilha e Desvio Final.....	17
3.4. Escalonador.....	17
3.4.1. Salvamento do Contexto Volátil no TCB.....	17
3.4.2. Captura do Ponto de Interrupção.....	17
3.4.3. Atualização do Estado do Sistema.....	18
3.4.5. Substituição do Frame na Pilha.....	18
3.4.6. Atualização do Alvo de Retorno.....	18
3.5 Configuração do Kernel.....	19
3.5.1. Objetos adicionados no Makefile.....	19
4. Testes realizados.....	20
5. Resultados.....	21
6. Conclusão.....	22

1. INTRODUÇÃO

O objetivo desta atividade foi evoluir o **microkernel** fornecido pela disciplina, originalmente baseado em escalonamento **cooperativo**, para suportar escalonamento **preemptivo** utilizando interrupções periódicas de timer. A implementação foi realizada sobre a arquitetura RISC-V, utilizando o ambiente **QEMU** para simulação e **OpenSBI** para programação dos eventos de timer.

Os principais recursos adicionados foram:

1. **Interrupções periódicas de timer;**
2. **Tratamento de traps em modo supervisor;**
3. **Escalonamento preemptivo;**
4. **Algoritmo de escalonamento Round-Robin.**

1.1 Materiais utilizados

Para desenvolvimento e execução da atividade foi utilizado o ambiente virtual **Codespace** do **GitHub**.

Os códigos base foram obtidos na documentação fornecida sobre o Microkernel.

2. DESENVOLVIMENTO

Inicialmente foi realizado o estudo do repositório **BasicMicrokernel**, analisando os seguintes módulos:

- **start.S** (processo de boot);
- **uart.c** e **uart.h** (comunicação serial);
- **memory.c** e **memory.h** (gerência de memória);
- **task.c** e **task.h** (criação de tarefas);
- **context.S** (troca de contexto);
- **scheduler.c** e **scheduler.h** (escalonamento cooperativo);
- **linker.ld** (organização da memória);
- **Makefile** (processo de compilação).

Também foram estudados os capítulos 14 e 15 da documentação, que apresentam a teoria e os exemplos necessários para implementação do sistema preemptivo.

2.1 Arquivos adicionados

- **include/timer.h**
- **kernel/timer.c**
- **include/trap.h**
- **kernel/trap.c**
- **boot/trap_entry.S**

2.2 Arquivos modificados

- **kernel/main.c**
- **include/scheduler.h**
- **kernel/scheduler.c**
- **Makefile**

3. IMPLEMENTAÇÃO

3.1. Módulo Timer

Foram criados os arquivos **timer.c** e **timer.h**. O módulo possui as seguintes responsabilidades:

- Configuração do intervalo entre interrupções;
- Programação do próximo evento de timer utilizando OpenSBI;
- Habilitação das interrupções de timer em modo supervisor.

A função **timer_init()** realiza a configuração inicial do temporizador e a função **timer_next()** agenda a próxima interrupção.

3.1.1. Arquivo timer.h

```
#ifndef TIMER_H
#define TIMER_H
#include <stdint.h>

void timer_init(uint64_t interval);
void timer_next(void);

#endif
```

A função **timer_init()** realiza a configuração inicial do temporizador e a função **timer_next()** agenda a próxima interrupção.

3.1.2. Chamada SBI timer

Implementação da chamada SBI timer

```
static inline void sbi_set_timer(uint64_t time)
{
    register unsigned long a0 __asm__ ("a0") = time;
    register unsigned long a6 __asm__ ("a6") = 0; //FID para set_timer
    register unsigned long a7 __asm__ ("a7") = 0x54494D45; // EID
    oficial da extensão "TIME"

    __asm__ volatile (
        "ecall"
        : "+r" (a0)
        : "r" (a6), "r" (a7)
        : "memory"
    );
}
```

A função **sbi_set_timer** é o mecanismo do microkernel (rodando em modo de privilégio, Supervisor ou **S-mode**) utiliza para solicitar que o firmware subjacente (o **OpenSBI**, que roda em Machine Mode ou **M-mode**) agende a próxima interrupção de hardware do cronômetro da CPU.

```
register unsigned long a0 __asm__ ("a0") = time;
```

- Recebe o parâmetro **time**. Ele indica o momento absoluto (em ciclos de clock da CPU) em que o alarme deve tocar.

```
register unsigned long a6 __asm__ ("a6") = 0;
```

- Recebe o valor **0**. Dentro da extensão **"TIME"**, a função de índice **0** é justamente a **set_timer**.

```
register unsigned long a7 __asm__ ("a7") = 0x54494D45;
```

- Recebe o valor hexadecimal **0x54494D45**. É o identificador padrão da extensão de gerenciamento de tempo da **SBI**.

```

__asm__ volatile (
    "ecall"
    : "+r" (a0)
    : "r" (a6), "r" (a7)
    : "memory"
);
}

```

- Esse trecho específico é utilizado para fazer uma **chamada de sistema para o firmware (OpenSBI)**, solicitando um serviço que o Kernel não consegue fazer sozinho (como programar o temporizador de hardware).
- A instrução **ecall (Environment Call)** é o gatilho que altera o modo de privilégio da CPU.
- **: "+r" (a0)** (Operandos de Saída)
 1. **+**: Significa que a variável **a0** é de **leitura e escrita** (o valor entra na instrução e depois é modificado por ela);
 2. **r**: Indica que o compilador deve mapear esta variável num registrador físico da CPU;
 3. O **OpenSBI** usa o registrador **a0** para devolver o código de erro ou sucesso da operação (geralmente **0** para sucesso).
- **: "r" (a6), "r" (a7)** (Operandos de Entrada)
 1. Informa-se ao compilador que os valores contidos nas variáveis **a6** (Function ID) e **a7** (Extension ID) devem estar devidamente carregados nos seus respectivos registradores físicos antes de a instrução **ecall** ser disparada.
- **: "memory"**
 1. Avisa ao compilador que a chamada externa ao SBI pode ler ou alterar a memória principal.

3.1.3. void timer_next(void)

```
void timer_next(void)
{
    uint64_t now;

    /* TODO: Ler o CSR time.      */
    __asm__ volatile ("csrr %0, time" : "=r" (now));

    /* TODO: Programar now + tick_interval.*/
    sbi_set_timer(now + tick_interval);
}
```

A função **timer_next** tem como responsabilidade primária o **reagendamento contínuo da próxima interrupção de temporizador** do sistema de forma preemptiva. Ela atua como o motor do escalonador de tempo (*Round-Robin*), garantindo que, após o estouro de um *tick* atual, um novo evento futuro seja programado na CPU.

```
uint64_t now;
```

- Uma variável local de **64 bits** é alocada para armazenar o valor absoluto do contador de ciclos de clock de tempo real da CPU no instante exato da execução.

```
__asm__ volatile ("csrr %0, time" : "=r" (now));
```

Nesta linha, o código utiliza **Assembly em Linha (Inline Assembly)** para ler um registrador interno especial do processador.

- **csrr (CSR Read)**: É a instrução em linguagem de máquina do RISC-V usada para ler Registradores de Controle e Estado (*Control and Status Registers*).
- **time**: É o CSR de hardware que incrementa continuamente em uma frequência fixa (no caso do ecossistema mapeado pelo **OpenSBI**, geralmente associado ao temporizador físico **mtime**). Ele funciona como o relógio biológico da CPU.
- **"=r" (now)**: Restrição de saída que instrui o compilador C a pegar o valor lido do registrador pelo hardware e depositá-lo diretamente na variável **now**.

```
sbi_set_timer(now + tick_interval);
```

O sistema calcula o momento exato em que o próximo alarme deve tocar através da fórmula matemática elementar:

$$\text{Tempo de Disparo} = \text{Tempo Atual (now)} + \text{Intervalo do Tick}$$

Esse valor resultante é passado como parâmetro para a função **sbi_set_timer**.

A correta execução da função **timer_next** é o que diferencia um sistema cooperativo (onde as tarefas precisam ceder o controle voluntariamente) de um **sistema preemptivo**.

3.1.4. void timer_init(void)

```
void timer_init(uint64_t interval)
{
    if (interval != 0)
        tick_interval = interval;

    timer_next();

    /* TODO:
     * Habilitar STIE no CSR sie. Habilitar SIE global em sstatus. */

    unsigned long sie_mask = (1ULL << 5); // STIE bit
    unsigned long sstatus_mask = (1ULL << 1); // SIE bit

    __asm__ volatile (
        "csrs sie, %0"
        :
        : "r" (sie_mask)
        : "memory"
    );

    __asm__ volatile (
        "csrs sstatus, %0"
        :
        : "r" (sstatus_mask)
        : "memory"
    );
}
```

Responsabilidades:

1. Armazenar o intervalo desejado;
2. Programar o primeiro evento de **timer**;
3. Habilitar a interrupção **STIE** no **CSR sie**;
4. Habilitar interrupções globais no **CSR sstatus**.

```
if (interval != 0)
    tick_interval = interval;
```

- O parâmetro **interval** dita quantos ciclos de clock da CPU devem decorrer entre cada interrupção de tempo. A função realiza uma validação de segurança: se o argumento for válido (diferente de zero), a variável estática global **tick_interval** é atualizada. Esse valor determina diretamente a "fatia de tempo" (*quantum*) que cada tarefa terá para rodar antes de sofrer a preempção.

```
timer_next();
```

- A função invoca imediatamente a rotina **timer_next()**. Isso faz com que o hardware leia o tempo absoluto atual do processador (**csrr time**) e adicione o **tick_interval** calculado no passo anterior, enviando uma requisição ao **OpenSBI** para armar o primeiro "alarme" de hardware.

```
unsigned long sie_mask = (1ULL << 5); // STIE bit
unsigned long sstatus_mask = (1ULL << 1); // SIE bit
```

Para interagir com os registradores internos da arquitetura RISC-V, é criado duas máscaras binárias usando operações de deslocamento de bits (*bit-shift*):

1. **sie_mask (Bit 5)**: O bit 5 do registrador **sie** (*Supervisor Interrupt Enable*) é o **STIE** (*Supervisor Timer Interrupt Enable*). Ativar esse bit avisa o processador que o kernel aceita receber interrupções originadas pelo temporizador.
2. **sstatus_mask (Bit 1)**: O bit 1 do registrador **sstatus** (*Supervisor Status*) é o **SIE** (*Supervisor Interrupt Enable global*). Ele funciona como a "chave geral" das interrupções no modo Supervisor. Se estiver zerado, nenhuma interrupção externa ou de timer chega ao kernel, agindo como uma blindagem protetora.

```

__asm__ volatile (
    "csrs sie, %0"
    :
    : "r" (sie_mask)
    : "memory"
);

```

O código utiliza Assembly em linha para injetar dados no registrador de hardware.

- **csrs (CSR Set):** Esta instrução executa uma operação lógica *OR* bit a bit entre o valor atual do registrador **sie** e a nossa máscara (**sie_mask**). Ela modifica **apenas** o bit número 5 para **1 (ativado)**, preservando a configuração de todos os outros bits do registrador.

```

__asm__ volatile (
    "csrs sstatus, %0"
    :
    : "r" (sstatus_mask)
    : "memory"
);

```

Seguindo a mesma lógica do passo anterior, a instrução **csrs** aplica a máscara **sstatus_mask** no registrador de estado global **sstatus**. Ao mudar o bit 1 para **1**, o processador efetivamente abre os canais de escuta para eventos assíncronos. A partir do momento em que essa instrução é finalizada, a CPU está oficialmente autorizada a desviar o fluxo para o tratador de interrupções assim que o alarme do timer bater.

3.2. Tratamento de Traps

Foram criados os arquivos **trap.c** e **trap.h**. O tratador de traps realiza:

1. Leitura do registrador **CSR scause**;
2. Verificação do tipo de exceção/interrupção;
3. Identificação da interrupção de timer;
4. Chamada da função **timer_next()**;
5. Chamada da função **schedule_from_trap()**.

Esse mecanismo permite que o kernel tome controle da CPU periodicamente para realizar a troca de tarefas.

3.2.1. Arquivo trap.h

```
#ifndef TRAP_H
#define TRAP_H
#include <stdint.h>

void trap_handler(uint64_t *frame);

#endif
```

3.2.2. Arquivo trap.c

```
#include "trap.h"
#include "timer.h"
#include "scheduler.h"
#include "uart.h"

extern void uart_print(const char*);

void trap_handler(uint64_t *frame)
{
    uint64_t scause;

    /* TODO: Ler CSR scause.*/
    asm volatile(
        "csrr %0, scause"
        : "=r"(scause)
    );

    // Isolamos o bit 63 (Interrupt) e os 4 bits inferiores
    (Exception Code)
    uint64_t is_interrupt = (scause >> 63) & 1ULL;
    uint64_t trap_code = scause & 0xFFFFULL;

    /* TODO: Verificar se interrupção.*/
    if (is_interrupt && (trap_code == 5)) { /* TODO: Identificar
    Supervisor Timer Interrupt.*/

        /* TODO: Chamar timer_next().*/
        timer_next();

        /*Chamar schedule_from_trap(frame).*/
        schedule_from_trap(frame);
    }
}
```

```

        return;
    }
    uart_print("Unhandled trap\n");
    while (1);
}

```

- **uart.h e extern void uart_print:** Fornecem acesso ao driver da porta serial UART para depuração de erros em terminal.

3.2.3. Leitura do CSR scause via Hardware

```

uint64_t scause;

asm volatile(
    "csrr %0, scause"
    : "=r" (scause)
);

```

Para descobrir o que gerou o desvio, o código consulta o registrador de controle mais importante da interrupção: o **scause** (*Supervisor Cause Register*).

- Através da instrução Assembly inline **csrr (CSR Read)**, o kernel extrai o valor de 64 bits de dentro do processador e o move para a variável C **scause**.

3.2.4. Decodificação de Causa (Bitmasking)

```

uint64_t is_interrupt = (scause >> 63) & 1ULL;
uint64_t trap_code = scause & 0xFFFFULL;

```

Na arquitetura RISC-V, o registrador **scause** armazena duas informações cruciais de forma compactada:

1. **O bit mais significativo (Bit 63):** Define se o evento foi uma *Interrupção* (causada por periféricos externos de forma assíncrona, valor 1) ou uma *Exceção* (causada por erro no próprio código em execução, valor 0). O código isola esse bit jogando-o 63 posições para a direita (>> 63).
2. **Os bits inferiores (0 a 11):** Contém o código numérico do evento (*Exception Code*). O código mascara os bits inferiores com 0xFFFF.

3.2.5. Filtro e Tratamento do Timer do Supervisor (STIP)

```
if (is_interrupt && (trap_code == 5)) { /* TODO: Identificar
Supervisor Timer Interrupt.*/

    /* TODO: Chamar timer_next().*/
    timer_next();

    /*Chamar schedule_from_trap(frame).*/
    schedule_from_trap(frame);
    return;
}
```

O bloco condicional filtra o evento com rigor matemático:

- **is_interrupt**: Deve ser igual a 1 (evento assíncrono).
- **trap_code == 5**: Na tabela oficial da arquitetura RISC-V, o código 5 mapeia estritamente a interrupção **STIP** (*Supervisor Timer Interrupt*), disparada pelo hardware do cronômetro da CPU.

Se a condição for verdadeira, o ciclo preemptivo avança:

1. **timer_next()**: Comunica-se com o OpenSBI para limpar a interrupção atual e agendar o momento exato do próximo *tick*.
2. **schedule_from_trap(frame)**: Passa o ponteiro da pilha (onde estão guardados os registradores da tarefa que foi interrompida) para o escalonador salvar o contexto atual e sobrescrevê-lo com os dados da próxima tarefa pronta.
3. **return**: Interrompe a execução da função imediatamente, retornando de volta para a penúltima linha do `trap_entry.S`, que desempilhará os dados modificados da nova tarefa.

3.2.6. Tratamento de Falhas Críticas (Fallback)

```
uart_print("Unhandled trap\n");
while (1);
```

Se o processador acionar o **trap_handler** por qualquer outro motivo que não seja o timer (por exemplo, se a tarefa tentar acessar um ponteiro nulo, gerando um *Load Page Fault*), o fluxo ignora o **if**, imprime o aviso de erro na tela e entra em loop infinito, paralisando o núcleo da CPU para evitar corrupção de memória.

3.3 Ponto de Entrada em Assembly

Foi criado o arquivo `trap_entry.S`.

Esse módulo é responsável por:

- Salvar todos os registradores da CPU;
- Chamar a função `trap_handler()`;
- Restaurar os registradores salvos;
- Retornar para a tarefa selecionada através da instrução `sret`.

A utilização de Assembly foi necessária para preservar completamente o contexto da tarefa interrompida.

3.3.1. Alocação do Trap Frame na Pilha

```
trap_entry:
    addi sp, sp, -256
```

As pilhas na arquitetura RISC-V crescem para baixo na memória. Esta instrução subtrai **256 bytes** do ponteiro de pilha (**sp**). Esse espaço de 256 bytes é chamado de **Trap Frame**. Como cada registrador em uma CPU de **64 bits** ocupa **8 bytes**, e existem 32 registradores gerais (**x0 a x31**), a conta é exata:

$$32 \text{ registradores} \times 8 \text{ bytes} = 256 \text{ bytes}$$

3.3.2. Salvamento do Stack Pointer Antigo

```
sd x1, 0(sp)
addi x1, sp, 256
sd x1, 8(sp)
ld x1, 0(sp)
```

- Salva provisoriamente o **x1 (ra)** no índice **0**;
- Faz **x1 = sp atual + 256** (calcula o valor exato do `sp` antigo);
- Salva esse `sp` antigo reconstruído no índice 1 (`8(sp)`);
- Restaura o valor original do `x1` de volta.

3.3.3. Salvamento em Massa dos Registradores Gerais

```
sd x1, 0(sp)
```

```
sd x3, 16(sp)
sd x4, 24(sp)
...
sd x31, 240(sp)
```

A instrução **sd** (*Store Doubleword*) copia o dado contido no registrador físico direto para uma posição da memória RAM (na pilha). O código varre a **CPU** salvando todos os registradores, pulando apenas o **x0** (que por hardware é sempre zero) e o **x2** (que é o **sp**, já tratado no bloco anterior).

3.3.4. Despacho para o Ambiente C

```
mv a0, sp
call trap_handler
```

Antes de chamar a função em **C**, a convenção de chamada dita que o primeiro argumento deve ser colocado no registrador **a0**.

- **mv a0, sp**: Copia o endereço do início do nosso frame de registradores na pilha para **a0**.
- Quando o `trap_handler(uint64_t *frame)` em **C** começar a rodar, o ponteiro ***frame** receberá exatamente esse endereço, permitindo que a função **schedule_from_trap** leia e modifique os registradores usando um laço em **C**.

3.3.5. Restauração de Contexto

```
ld x1, 0(sp)
ld x3, 16(sp)
...
ld x31, 240(sp)
```

A instrução **ld** (*Load Doubleword*) faz o inverso: retira o dado que está na **RAM** e injeta de volta nos registradores físicos da CPU. Se o escalonador substituiu o bloco, a **CPU** agora assume os valores da nova tarefa.

3.3.6. Restauração do Ponteiro da Pilha e Desvio Final

```
ld sp, 8(sp)
sret
```

- **ld sp, 8(sp)**: Restaura o ponteiro da pilha para o topo original da tarefa ativa (limpando o espaço de **256 bytes** usado temporariamente).
- **sret (Supervisor Return)**: É a instrução privilegiada que encerra o tratamento do trap. Ela faz a CPU copiar o endereço gravado no **CSR sepc** para o contador de

programa (**pc**) e reabilita automaticamente as interrupções de hardware locais, fazendo a nova tarefa rodar instantaneamente.

3.4. Escalonador

Os arquivos **scheduler.c** e **scheduler.h** foram modificados.

Foi implementada a função **schedule_from_trap()**, responsável por:

- Salvar o contexto da tarefa atual;
- Selecionar a próxima tarefa através do algoritmo Round-Robin;
- Restaurar o contexto da tarefa escolhida.

Diferentemente da versão original, a troca de contexto não depende mais da chamada explícita da função **yield()**.

3.4.1. Salvamento do Contexto Volátil no TCB

```
for(int i = 0; i < 32; i++){  
    tasks[prev].regs[i] = frame[i];  
}
```

O ponteiro ***frame** aponta para os 256 bytes da pilha onde o **trap_entry.S** acabou de despejar os registradores físicos da CPU. O laço **for** copia esses **32** valores de **64 bits** para dentro do vetor estruturado **regs[i]** da tarefa atual no array de TCBs (**tasks**).

3.4.2. Captura do Ponto de Interrupção

```
uint64_t current_sepc;  
asm volatile("csrr %0, sepc" : "=r"(current_sepc));  
tasks[prev].sepc = current_sepc;
```

Quando o hardware sofre uma interrupção, o processador salva automaticamente o endereço da instrução que foi interrompida em um CSR especial chamado **sepc** (*Supervisor Exception Program Counter*).

- O código faz a leitura desse CSR via Assembly inline (**csrr**).
- Salva esse endereço de memória no **TCB** da tarefa velha. Isso garante que, quando essa tarefa voltar a rodar no futuro, o kernel saberá exatamente de qual linha ela deve continuar.

3.4.3. Atualização do Estado do Sistema

```
current = next;
```

A variável global **current** é atualizada oficialmente, indicando que a nova tarefa passa a ser a dona do tempo de processador a partir daquele instante.

3.4.5. Substituição do Frame na Pilha

```
volatile uint64_t *dest_frame = frame;
for(int i = 0; i < 32; i++){
    dest_frame[i] = tasks[next].regs[i];
}
```

- O código cria um ponteiro local com a diretiva **volatile**. Isso força o compilador C a fazer acessos diretos e obrigatórios à memória física da **RAM**, impedindo-o de tentar otimizar a escrita guardando dados em registradores voláteis que serão destruídos na saída do trap.
- O laço **for** lê os registradores previamente guardados no **TCB** da **nova tarefa** (**tasks[next].regs[i]**) e os escreve **por cima** do frame antigo na pilha.
- Ao fazer isso, o contexto da tarefa velha é apagado da pilha atual e substituído integralmente pelos dados da nova tarefa.

3.4.6. Atualização do Alvo de Retorno

```
asm volatile("csrw sepc, %0" :: "r"(tasks[next].sepc));
```

Por fim, o código injeta o endereço de instrução salvo da nova tarefa diretamente de volta no registrador **sepc** via instrução **csrw** (*CSR Write*).

3.5 Configuração do Kernel

O arquivo **main.c** foi atualizado para:

- Configurar o registrador **CSR stvec** apontando para **trap_entry**;
- Inicializar o sistema de **timer**;
- Habilitar interrupções;
- Iniciar o escalonador.

Também foi necessário atualizar o **Makefile** para incluir os novos arquivos criados.

3.5.1. Objetos adicionados no Makefile

Foi adicionado os seguintes objetos:

- trap_entry.o
- trap.o
- timer.o

4. Testes realizados

Após concluir a implementação, todas as chamadas à função **yield()** foram removidas das tarefas de teste.

O kernel foi compilado e executado no **QEMU**.

A saída observada foi semelhante a:

```
=== Preemptive Kernel ===  
Task 1 running  
Task 2 running  
Task 1 running  
Task 2 running  
Task 1 running  
Task 2 running  
...
```

Observou-se que as tarefas continuaram alternando automaticamente, demonstrando que as interrupções periódicas estavam funcionando corretamente e que o escalonamento preemptivo foi implementado com sucesso.

5. Resultados

Os objetivos da atividade foram atingidos com sucesso.

A implementação permitiu que o microkernel:

- Recebesse interrupções periódicas de timer;
- Tratasse traps em modo supervisor;
- Realizasse troca de contexto automática;
- Executasse escalonamento preemptivo;
- Utilizasse o algoritmo Round-Robin para alternar entre tarefas.

A remoção das chamadas a **yield()** confirmou que a alternância entre tarefas passou a ocorrer por preempção, não mais por cooperação das **tasks**.

6. Conclusão

A atividade permitiu compreender o funcionamento interno de sistemas operacionais baseados em microkernel, especialmente os mecanismos de interrupção, tratamento de traps e escalonamento de processos.

A implementação do escalonamento preemptivo utilizando timer demonstrou como o sistema operacional pode recuperar o controle da CPU periodicamente, garantindo compartilhamento justo do processador entre múltiplas tarefas através do algoritmo Round-Robin.